Computer Networks 257 (2025) 110975

Contents lists available at ScienceDirect

Computer Networks

journal homepage: www.elsevier.com/locate/comnet

Enhanced detection of obfuscated HTTPS tunnel traffic using heterogeneous information network

Mengyan Liu, Gaopeng Gou, Gang Xiong, Junzheng Shi *, Zhong Guan, Hanwen Miao, Yang Li

Institute of Information Engineering, Chinese Academy of Sciences, Beijing, China
School of Cyber Security, University of Chinese Academy of Sciences, Beijing, China

ARTICLE INFO

Dataset link: HTTPS Tunneling Predictive Analysis Dataset (Original data)

Keywords:
Security protocol
HTTPS tunnel
Graph neural network

ABSTRACT

HTTPS tunnel-based VPN services are increasingly used for malicious activities, such as remote control and data exfiltration. As detection mechanisms improve, some adversaries employ obfuscation techniques to evade detection. However, existing research mainly focuses on identifying HTTPS tunnel traffic and lacks specific studies on obfuscated traffic. In this paper, we propose HINT, a novel method that transforms HTTPS tunnel traffic detection into a graph node classification problem. Specifically, we construct a heterogeneous information graph to model the connections between clients and the VPN services. To enrich the graph's semantics, we incorporate distinctive characteristics that are challenging to disguise and encapsulate them into specialized fingerprint nodes. Then we apply a hierarchical attention mechanism to automatically discern the significance of different nodes. Experimental results and extended analysis reveal that by integrating host topology, service statistics, and client traffic features, HINT maintains robust classification power when traffic shaping and padding techniques are employed. It is particularly effective without relying on packet sequences or payload information and maintains high detection capability even with added network noise.

1. Introduction

Traditional VPN protocols, including IPSec and OpenVPN, are vulnerable to security risks due to weak encryption [1–3] and implementation flaws [4–6], making them easily monitored and prone to leaking the transmitted information. As a result, cyber actors are increasingly turning to stealthier techniques to bypass traditional detection methods. The ubiquity of HTTPS, used by 85.9% of websites [7], makes it a good choice against cencorship. Since firewalls and security sensors often treat HTTPS traffic as legitimate, covert tunnels over HTTPS are increasingly used for cybercrime, cyber attacks, cyberterrorism, and other malicious activities [8–10]. A notable example was the Google Aurora attack, where attackers established HTTPS tunnels to the victims' hosts, enabling the theft of account information and persistent network penetration over several months, ultimately resulting in extensive data exfiltration. Thus, distinguishing malicious tunnel traffic from the large volume of legitimate HTTPS traffic becomes a major challenge.

Researchers have demonstrated considerable success in identifying HTTPS tunnel traffic, with methodologies falling into four primary categories. Some methods utilize plaintext features, such as certificates, to accurately identify types such as Tor and VPN traffic. Other methods design hand-crafted features, such as packet sizes and statistical features, and feed them to machine learning models to discern patterns. A third merges deep learning's capability for automated feature extraction from raw data with handpicked attributes. Recent advancements [11, 12] have enhanced precision by integrating host features, communication patterns, and active probing insights. However, emerging studies [13–16] show that traffic morphing and fingerprint obfuscation are effective ways to defend against traffic analysis. These obfuscation strategies, prevalent in VPNs and encrypted proxies, can markedly modify traffic characteristics, resulting in performance degradation for traffic-based detection methods. For instance, research [14] shows that random padding reduces the True Positive Rate (TPR) from 0.859 to 0.688. With the advanced padding mode of XTLS-vision, the TPR plummets even further to only 0.513. The significant performance loss caused by these evasion techniques highlights the need to explore methods for identifying obfuscated traffic.

To address the above challenges, in this paper, we propose the Heterogeneous Information Network for HTTPS Tunnel Traffic Detection

* Corresponding author at: School of Cyber Security, University of Chinese Academy of Sciences, Beijing, China.
E-mail addresses: liumengyan@iie.ac.cn (M. Liu), gougaopeng@iie.ac.cn (G. Gou), xionggang@iie.ac.cn (G. Xiong), shijunzheng@iie.ac.cn (J. Shi), guanzhong@iie.ac.cn (Z. Guan), miaohanwen@iie.ac.cn (H. Miao), liyang@iie.ac.cn (Y. Li).

https://doi.org/10.1016/j.comnet.2024.110975
Received 2 August 2024; Received in revised form 3 November 2024; Accepted 2 December 2024
Available online 6 December 2024
1389-1286/© 2024 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

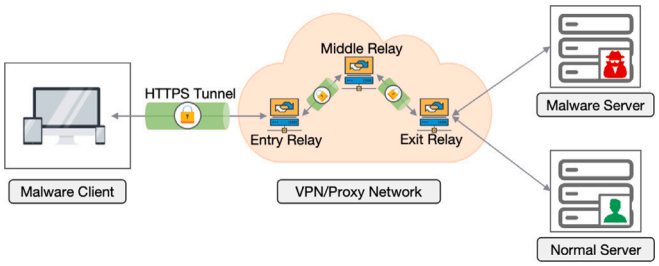


Fig. 1. Process of achieving anonymity in communication with VPN or proxy.

(HINT) method—an approach that shifts from conventional packet-level and flow-level analyses to identify HTTPS tunnel traffic, when traffic shaping and padding techniques are employed. In detail, by analyzing extensive communication flows, we extract critical insights to build a heterogeneous graph that captures various user interactions and the distribution patterns of VPN services, all represented within a network of interconnected nodes and edges. To enrich the graph's semantic information, we incorporate novel characteristics that are challenging to disguise and encapsulate them into specialized fingerprint nodes. Then, we adopt a hierarchical attention mechanism to automatically learn the importance of different types of nodes and meta-paths, combining features from the host, service, and client levels. Finally, we implement node classification using a classifier, thus transforming the HTTPS tunnel traffic detection task into a graph node classification problem.

In summary, the contributions of this work are:

- We propose HINT, a novel method for detecting HTTPS tunnel traffic. HINT combines user communication patterns with VPN distribution characteristics to construct a knowledge graph, transforming the detection task into a graph node classification problem for enhanced accuracy.
- To enrich graph semantics, we add hard-to-disguise characteristics as specialized fingerprint nodes, including traffic features, statistical metrics, burst features, latency, and information entropy.
- We apply a dual-level attention mechanism to strengthen detection capability by combining features at the host, service, and client levels.
- Our extensive experiments on self-built datasets demonstrate that HINT remains robust against obfuscation techniques. Notably, it achieves high accuracy without relying on packet sequences or payload information.

The remainder of the paper is organized as follows. Section 2 introduces the background of our work. Section 3 reviews some related works. Section 4 highlights the overall design of HINT and Section 5 presents the main setting for experiments. Section 6 illustrates the experiments results. Section 7 provides the discussion and Section 8 concludes the paper.

2. Background

This section briefly introduces VPN traffic in HTTPS tunnels, including the principles of HTTPS tunneling and countermeasures against HTTPS Tunnel traffic fingerprinting.

2.1. The principle of HTTPS tunneling

HTTPS tunneling encapsulates original protocol data within the HTTPS protocol for secure communication. As shown in Fig. 1, there are three entities: users with VPN or proxy clients, VPN or proxy services operating on cloud platforms, and the destination services.

Table 1
Comparison of network obfuscation approaches and fingerprint obfuscation techniques for VPN and Proxy.

Tool	Approach	Obfuscation techniques
Tor Meek	tunneling	Traffic Shaping
Tor obfs	randomizer	Traffic padding, Traffic Shaping
MTProxy	tunneling	Traffic padding
Psiphon	tunneling	Traffic padding
uProxy	tunneling	Traffic padding
IMProxy	tunneling	Traffic Shaping
NordVPN	tunneling	Traffic Shaping
HideIPVPN	tunneling	Traffic Shaping
Shadow VPN	tunneling	Traffic padding
Xray	tunneling	Traffic padding
SSL-VPN	tunneling	Traffic Shaping
IPsec-VPN	tunneling	Traffic Shaping

Users utilize SSL/TLS for secure authentication and encryption when connecting to the VPN or proxy service. These tools may use certificates from any website to forge seemingly legitimate HTTPS connections, making the data indistinguishable from normal web traffic. Once the VPN or proxy service receives the encrypted data, it decrypts the contents and relays the request to the final destination service. This form of HTTPS tunneling is widely used by VPNs. Similarly, Tor's Meek plugin [17] uses an HTTPS tunnel to avoid detection. Open-source tools supporting these technologies include ShadowsocksR, V2Ray, and Xray. Additionally, remote control malware such as CobaltStrike [18] often relies on HTTPS tunnels for its communication channels.

2.2. Obfuscation of HTTPS tunnel fingerprints

As mentioned earlier, it is possible to accurately classify encrypted traffic based on TLS fingerprint, packet sizes and packet inter-arrival times. To counteract this, techniques such as TLS fingerprint obfuscation, traffic padding, and shaping have been developed. However, our method does not depend on plaintext from the TLS handshake, so TLS fingerprint obfuscation is not a primary concern. Traffic padding and shaping modify packet sizes and packet inter-arrival times, effectively hiding user activities and preventing most inference attacks based on these features [15,19,20]. Following, we detail these techniques.

- Traffic padding technique involves appending extra bytes to original packets, standardizing the length of every incoming and outgoing packet to a fixed measure or a predetermined distribution [14,21]. When a packet falls short of the set length, the node will pad it with redundant information to reach the required size. It can defend against packet and traffic analysis attacks, thereby protecting the anonymity of sender and receiver.
- Traffic shaping technique models traffic by interspersing additional cover packets [21,22]. Client injects extra packets at predetermined probabilities or specified locations. A unique bit flag is included in the encrypted contents or header of the packet, indicating whether the packet is associated with real traffic or if it is a cover packet, which should be ignored by servers. However, detection models cannot distinguish between real and cover packets.

In Table 1, we summarize a range of typical VPN and proxy tools that employ obfuscation approaches and compare them qualitatively based on their fingerprint obfuscation methods.

3. Related work

There has been a growing body of research on the detection of HTTPS tunnel traffic. The fingerprint matching approach, similar to deep packet detection, relies on plaintext TLS handshake information. Previous studies [23–27] have extracted unencrypted features

from the network, transport, and application layers, such as domain names, certificates, cipher suites, and server IP addresses, to identify TOR and VPN traffic. When attackers disguise key information during the TLS handshake, these methods become ineffective. To address this, several studies [14,28,29] have explored machine learning for identifying HTTPS tunnel traffic using traffic sequences and statistical features. Further research [18] describes a method that filters HTTPS tunnel traffic using burst detection and applies machine learning for feature analysis with a voting-based outcome. However, these machine learning methods face challenges when traffic obfuscation techniques obscure packet sizes and timings features.

Deep learning simplifies feature extraction by automating it from raw traffic. For example, studies [10,30,31] employ Convolutional Neural Networks (CNNs) to classify encrypted VPN traffic by converting network packet features, such as payload lengths and traffic bytes, into image representations for improved accuracy. Some researchers [32] use transfer learning to detect obfuscated Tor traffic by fine-tuning a pre-trained Deep Neural Network (DNN) on a small Tor dataset. Recent studies [8,11,12,18] pivot to host-based methods, continuously analyzing flow data over time for more accurate classification, such as Wang et al. [11] introduce VPNChecker, which combines active probing with node communication relationship information to detect VPN servers in real-world scenarios, and Wails et al. [12] combine deep learning with host-based detection to identify Tor traffic more precisely. However, these methods may be vulnerable to adversarial attacks that introduce subtle perturbations to the original traffic, and the responses from active probing are now similar to those of normal web services.

Recent advances in obfuscated traffic detection highlight the use of feature extraction, autoencoders, Generative Adversarial Networks (GANs), and other deep learning techniques to address these challenges. In terms of feature extraction, some studies focus on identifying traffic characteristics that are difficult to obfuscate. For example, a study [21] utilizes the full distribution of packet sizes, rather than common statistics like mean and variance, to detect encrypted IoT traffic. Study [14] exploits the shared characteristic of nested protocol stacks to detect obfuscated proxy traffic, even when encryption and random padding are applied. Studies [16,33] extract statistical features from obfuscated Tor traffic, effectively overcoming the challenges posed by random padding and timing sequence randomization. Furthermore, with the development of deep learning, several studies [34–37] have employed generative architectures, such as autoencoders and GANs, to detect and recover mutated traffic through adversarial sample generation and denoising.

4. Methodology

In this section, we propose a novel method for HTTPS tunnel traffic detection named HINT. We first present the underlying motivation for the proposed method, then we delve into an exhaustive explanation of the method itself.

4.1. Motivation

VPNs and proxies have become indispensable tools in today's interconnected world, they offer secure communication over public networks. During the VPN communication process, the VPN or proxy typically exhibits the following traits:

- **Multi-port Configuration**. To enhance network efficiency and ensure accessibility and reliability, VPNs and proxies are commonly set up to monitor several ports under a single IP address. For example, OpenVPN can listen on both TCP and UDP ports, which helps bypass restrictions that could block certain ports or protocols. VPN services from identical vendors display discernible patterns, such as sequential port numbers. This tendency is likely due to vendors using automated scripts for large-scale deployment.

Table 2
Node types, categories, labels, and attributes in session-level subgraph.

Node type	Category	Label	Node attribute
Client node	IP	C	Client IP
Service node	IP and Port	S	Server IP,Port
Client Fingerprint	Burst characteristics	F_{c1}	Max flows/burst
		F_{c2}	Avg interval/burst
		F_{c3}	Max interval/burst
		F_{c4}	Min interval/burst
	Traffic characteristic	F_{c5}	Flow characteristic

- **Distinctive Connection Patterns**. Standard web services exhibit intricate connection patterns, largely due to the wide variety of clients that access them frequently, resulting in a diverse and complex network activity. This level of complexity is seldom observed with VPN services.
- **Dynamic VPN Client Connectivity**. The VPN client connects to multiple service endpoints, prioritizing them based on response times. It establishes a secure tunnel with the selected service and switches to alternatives if needed. The client maintains active connections with several VPN services during communication [11].

These phenomena motivate us to create knowledge graphs based on behavioral traits, specifically tracing user interactions with VPN services and isolating their distinctive characteristics. As shown in Fig. 2, HINT includes two steps. First, we gathered network traffic data relating to the external communications of multiple users over a specified period, and we used a heterogeneous graph [38] to model these communications. Then, HINT leverages Dual-level attention to learn the importance of nodes, and embeds it to the heterogeneous graph for HTTPS tunnel traffic classification. The process is further illustrated in the following sections.

4.2. Knowledge graph construction and semantic enhancing

We first introduce the knowledge graph we constructed for communication flows, which enables the integration of additional information and captures the rich relations among host, service, and client information. Each knowledge graph is partitioned into two types of subgraphs: session-level and host-level. We then merge these subgraphs into a complete knowledge graph.

4.2.1. Session-level subgraph

Given an observation time window t, For each network service, we construct the $G_s = (V, E)$ based on the communication flow between clients and the services, containing one service $S = \{s_1\}$, clients $C = \{c_1, \ldots, c_K\}$, and client fingerprints $F_c = \{f_{c1}, \ldots, f_{cn}\}$ as nodes, i.e., $V = S \cup C \cup F_c$. The set of edges E represents their relations. The details of constructing the network are described as follows.

First, for each network service, an edge is formed between a client node c and the service node s to represent the client's access to the service. When multiple clients interact with the same service, separate client nodes are created for each unique client-service interaction. To enhance the graph's semantic richness, we extract additional knowledge from the access flows and encapsulate them into dedicated fingerprint nodes f. Fingerprint nodes embody distinctive features in the network traffic. For each client node c, traffic features are extracted from the interactions between the client and the service, if a client includes one of these features, an edge is established between the client node c and the client-fingerprint node f_c in the graph. Each node is characterized by unique attribute features, as delineated in Table 2. Client-fingerprint nodes can be classified into two primary categories: burst characteristics and traffic characteristics. We then provide a detailed description of the methodology used in constructing the Client-Fingerprint nodes.

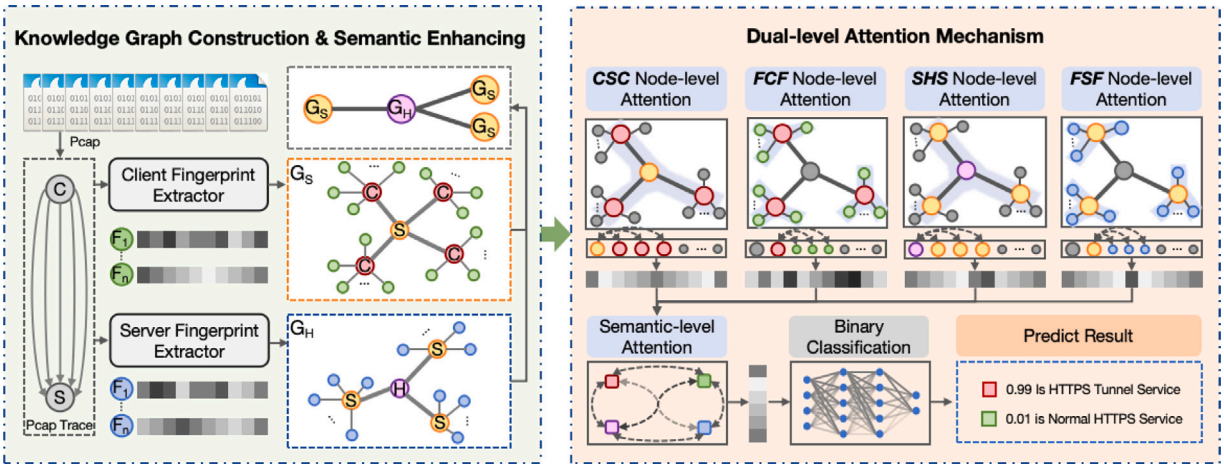


Fig. 2. The overall architecture of HINT.

- **Burst Characteristics.** Prior work in MTBD has demonstrated that the burst characteristics of HTTPS tunnel traffic have significant distinguishability [18]. Unlike normal services, when users or malware use HTTPS tunnels to communicate with other services, the client establishes multiple TCP flows to the tunnel server and the number of flows is significantly larger than the normal HTTPS service in a certain time window. Bursts refer to sequences of consecutive flows traveling within a short time. Specifically, given an original TCP flow record and the corresponding inter-arrival times, we first estimate the connection's Round-Trip Time (RTT) as the time elapsed between the observation of the SYN packet and the arrival of the first ACK packet sent from the client. When the start time of two consecutive flows is more than four times the estimated RTT, we split them into two separate burst flow sequences. Then, we analyze core statistical information based on bursts, such as the average and maximum value of interval time of flows per burst, the number of flows per burst, and other relevant metrics.
- **Traffic Characteristic.** We implemented a CNN–LSTM model to automatically extract potential communication features from network traffic data. Initially, a one-dimensional CNN extracts flow-level attributes, followed by a Long Short-Term Memory network (LSTM) that generates a vectorized fingerprint representing the characteristics of all communication flows. This hybrid approach merges spatial and temporal dimensions of network communication characteristics, enhancing the interpretation of intricate traffic patterns and their dynamic nature. Specifically, for multiple client-initiated communication flows directed towards a service within a specific time window t, each flow trace is characterized as a sequence of tuples <timestamp, ±packet_size>, where the sign of packet_size indicates the packet direction: positive for outgoing and negative for incoming. As depicted in Fig. 3, the input data is processed by a one-dimensional convolutional layer with 16 channels, followed by another with 32 channels, both using ReLU activation. A global average pooling layer then condenses these features. Then, the feature set is fed into a bi-directional LSTM network, designed to handle sequential data by learning dependencies and patterns over time from both directions. Dropout regularization is applied after the LSTM to prevent overfitting. Finally, the model uses two fully connected layers: the first maps LSTM outputs to a 64-dimensional feature space, and the second projects these features onto the output space to categorize the input data.

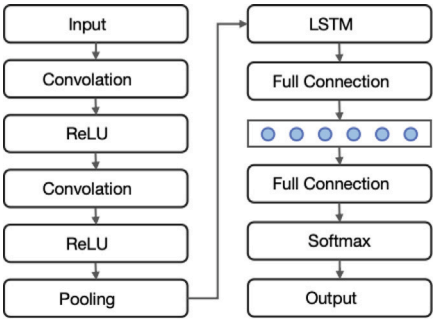


Fig. 3. Architecture of the CNN–LSTM model for traffic characteristics extraction.

4.2.2. Host-level subgraph

VPN often hosts similar services across different ports, which is likely caused by vendors' use of automated scripts for massive deployment. Motivated by this pattern, we propose the construction of a host-level graph to delineate the interrelations among services on a single host. For each host, we construct a graph $G_h = (V, E)$ that captures its service-related communications within a specific time window t. This graph comprises nodes representing the host $H = \{h_1\}$, its services $S = \{s_1, \ldots, s_K\}$, and their corresponding fingerprints $F_s = \{f_{s1}, \ldots, f_{sn}\}$, i.e., $V = H \cup S \cup F_s$. The edge set E captures the connectivity among these entities.

The host node refers to a computer or server that can provide various services, such as web services, email services, database services, file-sharing services, etc. Service nodes represent specific services deployed on the host. Service-Fingerprint nodes are indicative of unique statistical attributes and topological characteristics inherent in the services. When multiple services exist on a single host, the host node forms connections with several service nodes. These Service-Fingerprint nodes are in turn linked to their respective service nodes, signifying the characteristic features of the services. As outlined in Table 3, we enumerate the distinct attribute features for each node category. Similarly to Client-Fingerprint nodes, The Service-Fingerprint node's attributes help enhance semantics of the host-level subgraph, which are derived from statistics characteristics extractor. Statistics characteristics provide a comprehensive analysis of service traffic features and behaviors, and they can be categorized into three main types.

- **Statistics Characteristics.** Based on the service, we regard the flow as the basic statistical unit. We not only count the total number of flows but also measure the number of packets and bytes of each flow within the time window. Outgoing/Incoming

Table 3
Node types, categories, labels, and attributes in host-level subgraph.

Node type	Category	Label	Node attribute
Host node	IP	H	Server IP
Service node	IP and Port	S	Server IP, Port
Service Fingerprint	Statistics characteristics	F_{s1}	Outgoing pkts Ratio
		F_{s2}	Outgoing bytes Ratio
		F_{s3}	Incoming pkts Ratio
		F_{s4}	Incoming bytes Ratio
		F_{s5}	Avg pkts/flow
		F_{s6}	Avg bytes/flow
		F_{s7}	Client Count
		F_{s8}	Domain Count
	Latency characteristics	F_{s9}	Avg RTT Ratio
		F_{s10}	P25 RTT Ratio
		F_{s11}	P50 RTT Ratio
		F_{s12}	P75 RTT Ratio
	Entropy characteristic	F_{s13}	Information Entropy

pkts/bytes Ratio is a metric that tracks the percentage of outbound and inbound packets and bytes. Avg pkts/bytes per flow quantifies the typical traffic volume per flow. Additionally, we extracted the number of unique clients and the number of unique domains as characteristics.

- **Latency Characteristics.** Prior work [39] introduces a detection method for resip, which relies on assessing the difference in RTT between TLS and TCP packet exchanges. This approach is equally applicable to the identification of HTTPS tunnel traffic. Consequently, we have analyzed the ratio of TLS RTT to TCP RTT and the distribution of this metric.
- **Entropy Characteristic.** Furthermore, compared to conventional web services, the traffic handled by VPN services is typically more complex in terms of structure and content. VPNs facilitate the transmission of a diverse array of data types including emails, files, and real-time audio-video communications, whereas traditional web services are primarily oriented towards HTTP/HTTPS traffic, that is, web page content. Therefore, we employ the entropy of traffic to measure the complexity and diversity of traffic patterns.

4.2.3. Subgraphs fusion

By leveraging the correspondence between the service nodes S in the host-level subgraph and the service nodes S in the session-level subgraph, we merge the session-level subgraphs and the host-level subgraphs into a complete knowledge graph. Other types of nodes and edges are kept unchanged, following the structure of the original subgraph.

In a heterogeneous graph, nodes can be connected via different semantic paths, which are called meta-paths [40]. To denote the neighbor relationship of different semantics, we propose four types of meta-paths to connect four types of nodes in the knowledge graph:

- SHS meta-path: Connects a host node h with multiple service nodes s, indicating that multiple services are deployed on a single host.
- CSC meta-path: Connects a service node s with multiple client nodes c, denoting communication activities between the service and various clients.
- FCF meta-path: Connects a client node c with multiple feature nodes f_c, representing a range of communication behavior characteristics of the client.
- FSF meta-path: Connects a service node s with multiple feature nodes f_s, signifying a variety of feature information about the service.

4.3. Dual-level attention mechanism

HINT is a deep learning framework, which exploits the Heterogeneous Graph Attention Network to identify HTTPS Tunnel services based on a dual-level attention mechanism. The architecture could be divided into three objectives: node-level attention, semantic-level attention and binary classification layer.

4.3.1. Node-level attention

Node-level attention aims to learn the importance of meta-path based neighbors and assign different attention values to them [38]. For each knowledge graph, the node-level attention first learns the weights of meta-path-based neighbors and aggregates them to get the semantic-specific node embedding. Given $N_{\Phi}(u)$ denotes the meta-path Φ-based neighbors of node u (including itself) and node $v \in N_{\Phi}(u)$, the importance of the meta-path-based node pair $\langle u, v\rangle$ can be formulated as follows:

$$e_{uv}^{\Phi} = \sigma\left(a_{\Phi}^{T} \cdot \left[h_u \| h_v\right]\right) \tag{1}$$

where h_u and h_v are the features of node u and v, a_{Φ} is the node-level attention vector for meta-path Φ, σ denotes the activation function, and $\|$ denotes the concatenate operation, e_{uv}^{Φ} means how important node u will be for node v. After obtaining the importance between meta-path based node pairs, we normalize them to get the weight coefficient α_{uv}^{Φ} via softmax function:

$$\alpha_{uv}^{\Phi} = \mathrm{softmax}_v\left(e_{uv}^{\Phi}\right) = \frac{\exp\left(e_{uv}^{\Phi}\right)}{\sum_{k \in N_{\Phi}(u)} \exp\left(e_{uk}^{\Phi}\right)} \tag{2}$$

Then, the meta-path-based embedding of node u can be obtained by aggregating all neighbor attributes with the corresponding coefficients as follows:

$$z_u^{\Phi} = \Big\|_{k=1}^{K} \sigma\left(\sum_{v \in N_{\Phi}(u)} \alpha_{uv}^{\Phi} \cdot h_v\right) \tag{3}$$

where z_u^{Φ} is the learned embedding of node u for the meta-path Φ, K is the head number used by the multi-head attention mechanism [41]. Among the four types of meta-path in our work: $M = \{\Phi_{\mathrm{FCF}}, \Phi_{\mathrm{FSF}}, \Phi_{\mathrm{CSC}}, \Phi_{\mathrm{SHS}}\}$, after feeding node features into node-level attention, we can obtain four groups of semantic-specific node embeddings, denoted as $z_u^{\Phi_i}$, where $\Phi_i \in M$

4.3.2. Semantic-level attention

Semantic-level attention aims to learn the importance of each meta-path and assign proper weights to them [38]. Generally, every node in a heterogeneous graph contains multiple types of semantic information and semantic-specific node embedding can only reflect node from one aspect. To learn a more comprehensive node embedding, we use the semantic-level attention to automatically learn the importance of four types of semantic-specific embeddings and fuse them as a comprehensive node embedding. The importance of meta-path Φ_i based embedding is shown as follows:

$$w_{\Phi_i} = \frac{1}{|V|} \sum_{u \in V} \mathbf{p}^{T} \cdot \tanh\left(W \cdot z_u^{\Phi_i} + b\right) \tag{4}$$

where W is the weight matrix, b is the bias vector, and P is the semantic-level attention parameter matrix. V is the node set of the input knowledge graph, since we average the importance of all the semantic-specific node embeddings. Notably, all above parameters are shared for all meta-paths and semantic-specific embedding. After obtaining the importance of each meta-path, we normalize them via softmax function. The weight coefficient of meta-path Φ_i, denoted as β_{Φ_i}, can be obtained by normalizing the above importance of all meta-paths using softmax function,

$$\beta_{\Phi_i} = \mathrm{softmax}_i(w_{\Phi_i}) = \frac{\exp(w_{\Phi_i})}{\sum_{\Phi_i \in M} \exp(w_{\Phi_i})} \tag{5}$$

With the learned weights of each semantic-specific embedding, the comprehensive embedding Z_u of node u could be:

$$Z_u = \sum_{\Phi_i \in \mathcal{M}} \beta_{\Phi_i} \cdot z_u^{\Phi_i} \tag{6}$$

The comprehensive embeddings are the final representations of nodes learned by HINT, which aggregates multiple semantic characteristics. For instance, the service node S finally obtains the embedding with the semantics of host-level embedding $z_s^{\Phi_{\text{SHS}}}$, service-level embedding $z_s^{\Phi_{\text{FSF}}}$ and client-level embedding $z_s^{\Phi_{\text{CSC}}}$, $z_s^{\Phi_{\text{FCF}}}$. Then we apply the final embedding Z_S to HTTPS Tunnel detection task.

4.3.3. Binary classification

For the target service node S, we obtain the comprehensive embedding from the heterogeneous graph using the methods described above. Subsequently, we use the formula $Y(s)$ to enhance the feature, where R represents a linear layer containing learnable parameters w and b, and $R_*(s)$ denotes a linear layer with its own set of learnable parameters w and b. Finally, we use $L(s)$ to predict service node S.

$$\mathcal{R}(x) = \mathcal{W}x + b \tag{7}$$

$$\mathcal{Y}(s) = \mathcal{R}_1 \tanh \mathcal{R}_2(\mathcal{Z}_s) \tag{8}$$

$$\mathcal{L}(s) = \text{Softmax}(\mathcal{R}_3 \text{ReLu}(\mathcal{R}_4(\mathcal{Y}(s)))) \tag{9}$$

5. Experiment setup and implementation

This section describes our dataset collection and composition, the compared baselines, evaluation metrics, and the model implementation.

5.1. Dataset

The HINT method focuses primarily on the traffic characteristics, topology features, and long-term statistical features of VPN services, but existing datasets either lack this information or provide insufficient samples when transformed to host level, hindering the evaluation of our model's detection capabilities. Consequently, we use self-collected datasets for model assessment, consisting of one real dataset and two manually obfuscated datasets.

HTTPS tunneling dataset. We collected the traffic data from well-known VPN services, including Hotspot Shield Free, Browsec VPN, ZenMate VPN, Hoxx VPN, and ShadowsocksR which use the HTTPS tunneling for communication [23]. We developed a crawler script to automate visiting websites through VPNs. Specifically, clients (computers and mobile devices) connected to VPN servers with configured client software, and the crawler used Chrome to visit random sites from the Alexa top 10,000. To simulate realistic user behavior patterns, the crawler script was designed to browse at preset intervals with random pauses, closely mimicking human interaction habits. All traffic was routed through a configured router to a storage server, saving packets in pcap files. The entire process was repeated multiple times. Additionally, we gathered normal HTTPS traffic as negative instances from a corporate network of over a hundred users, which lacked explicit traffic labels. Five volunteers documented their online activities over a month without using VPNs or proxies, allowing us to record their service IPs and ports as non-VPN traffic. We named this dataset HTPA(HTTPS Tunneling Predictive Analysis).

As shown in Table 4, HTPA dataset includes two types of traffic: HTTPS tunnel traffic and normal HTTPS traffic. To ensure the validity of training and testing, we ensured that the data from any given host did not appear in both the training and testing sets. We have made the dataset open-source, and the download link for the dataset is: https-tunneling-predictive-analysis-dataset. To our knowledge, this is currently the largest dataset available on HTTPS tunnel.

Table 4
Summary of HTPA dataset: Distribution of hosts, services, flow sessions, and average clients per service.

Type	Hosts	Services	Flows	Avg Clients/Svc
Tunnel	623	1356	171 143	1.1
Normal	3648	3711	1 154 701	27.5
Total	4271	5067	1 325 844	/

Obfuscated dataset with traffic padding. To evaluate the method's robustness against traffic padding attacks, we generated a specialized interference dataset. Seeking experimental efficiency, we directly manipulated session logs within the HTPA dataset and altered packet sizes to simulate real-world conditions. Existing research [14] has shown that although simple random padding schemes, where the padding size of packets is randomly selected from a limited range, do indeed increase the challenge of detection, they do not render it impossible. Therefore, we opted for a more aggressive padding scheme. Given that fingerprint characteristics during the TLS handshake phase are pivotal for identifying tunneling traffic, we padded all packets according to a specific distribution, extending the length of each short packet to between 900 and 1400 bytes during the encapsulated TLS handshake phase [42]. Notably, modifications were applied solely to the tunneling traffic data, while normal HTTPS traffic data remained unchanged. We designated this adjusted dataset as HTPA-RD.

Obfuscated dataset with traffic shaping. We also constructed an obfuscated dataset that incorporates traffic shaping. To obscure the fingerprint characteristics, we injected decoy packets into outgoing direction during the initial TLS handshake and subsequent encapsulated TLS handshake processes. Specifically, decoy packets were randomly interspersed within the first ten packets of the upstream flow trace, and we calculated the timestamp of decoy packet as the average of the timestamps from its two adjacent packets. The size of each decoy packet was randomly selected between 0 and 1500 bytes, necessitating corresponding adjustments to flow statistics, including packet counts and byte sizes. This modified dataset was designated as HTPA-TS-n, where n spans from 1 to 3, indicating the number of decoy packets. Additionally, to further explore how the insertion position of decoy packet influences model performance, we placed a decoy packet at various predetermined locations in the upstream direction. These additional datasets were systematically denoted as Pos-n, with n ranging from 0 to 5, to signify their respective insertion positions.

5.2. Baselines

We implement six representative methods to compare with HINT. Specifically, we implemented the following methods:

- **AppScanner**: [43] builds traffic fingerprints using packet sequence information and their statistical features, combined with machine learning techniques.
- **DF**: [44] utilizes CNN to automatically extract traffic fingerprint features, aiming to perform website fingerprinting attacks on Tor tunnel traffic.
- **GraphDAPP**: [45] transforms the encrypted traffic classification problem into a graph classification problem.
- **MTBD**: [18] is an algorithm that detects HTTPS tunnels by using models built on flow, host, and packet information to make accurate decisions.
- **TLSFP**: [14] identifies the existence of HTTPS tunnel proxy traffic by detecting TLS handshakes fingerprint within the tunnel.
- **HBA**: [12] combines a deep learning-based classifier with a host-based detection strategy to address the challenge of accurately detecting Tor traffic in real-world networks.

5.3. Evaluation metrics

In this paper, we use four metrics for a comprehensive evaluation of different methods' performance in detecting HTTPS tunnel traffic: (1) True Positive Rate (TPR), which is the proportion of actual HTTPS tunnel traffic correctly identified by our model. (2) False Positive Rate (FPR), which is the proportion of regular traffic incorrectly labeled as HTTPS tunnel traffic by our model. (3) Area Under Curve (AUC), The AUC metric calculates the area under the ROC curve formed by TPRs and FPRs with multiple thresholds, which is frequently used in binary classification tasks. (4) Accuracy (ACC), The proportion of correctly classified samples to the total number of samples.

5.4. Implementation

For the proposed HINT, we randomly initialize parameters and employ Adam [46] optimization with a learning rate of 0.01 to iteratively update our model parameters. The learning schedule is defined using a StepLR scheduler that decays the learning rate by a factor of 0.1 after half the total epochs. We set the dimension of Node Embedding to 64 and the dimension of the semantic-level attention vector to 8. The number of attention heads is set to 8, and the dropout rate for attention is set to 0.2. The training process is envisioned to unfold across 40 epochs. For our CNN–LSTM model, we begin training using default initial parameters and employ Adam optimization with a fine-tuned learning rate of 0.001. Our convolutional layers utilize kernel sizes of 3 and 5 with corresponding paddings, while the LSTM has a 64-unit hidden layer and two stacked recurrent layers for temporal pattern recognition. To mitigate overfitting, we integrate a dropout rate of 0.5 in both the LSTM and fully connected layers.

6. Experiment results and evaluation

In this section, we evaluate our proposed method using self-collected datasets and assess the robustness of HINT against traffic padding and shaping. Then we examine the results of the hierarchical attention mechanism, conduct ablation studies to identify key model components, analyze parameter sensitivity, and evaluate time complexity.

6.1. Evaluate plain HTTPS tunnel traffic

We trained and tested our classifiers using the HTPA dataset, focusing on flow-level analysis rather than service-level analysis to ensure comprehensive fairness. This approach means that any misclassification of a node leads to all traffic associated with that service being considered misidentified. We can derive the following significant conclusions from Table 5.

HINT outperforms other models across nearly all evaluation metrics. While baselines like AppScanner and HBA also show strong results, HINT's perfect TPR and near-zero FPR highlight its superiority, indicating that every HTTPS tunnel service is accurately identified. We also implemented a CNN–LSTM model to automatically extract potential communication features from network traffic data, and it shows formidable classification capability. Although HINT demonstrates a lower FPR than the baselines, our evaluation reveals challenges in classifying web services with fewer than 20 flows and non-standard ports. Specifically, eight normal web services were misclassified as tunnel services. To enhance model performance, we will use an expanded training dataset and employ data augmentation techniques.

Table 5
Comparison results on HTPA dataset at flow level.

Method name	ACC	TPR	FPR	AUC
AppScanner	0.9889	0.9643	0.0037	0.9972
DF	0.9729	0.9593	0.0279	0.9931
GraphDAPP	0.9458	0.9519	0.0559	0.9725
MTBD	0.9615	0.9425	0.0172	0.9873
TLSFP	0.9342	0.9891	0.1052	0.9284
HBA	0.9736	0.9748	0.0267	0.9931
CNN–LSTM	0.9784	0.9553	0.0054	0.9896
HINT	**0.9988**	**1.0000**	**0.0004**	**0.9989**

Table 6
Comparison results on HTPA-RD dataset at flow level.

Method name	ACC	TPR	FPR	AUC
AppScanner	0.9299	0.9438	0.1539	0.9681
DF	0.9020	0.9457	0.1287	0.9473
GraphDAPP	0.9008	0.9488	0.1350	0.9872
MTBD	0.9423	0.9187	0.0178	0.9929
TLSFP	0.7120	0.9291	0.5042	0.7124
HBA	0.9253	0.9513	0.0835	0.9879
CNN–LSTM	0.9554	0.9139	0.0087	0.9758
HINT	**0.9984**	**0.9951**	**0.0006**	**0.9998**

6.2. Evaluate traffic padding effects

We evaluate the efficacy of HINT and baselines against the altered HTPA-RD dataset, which adds additional bytes to pcaps. Table 6 displays comparison results. We can derive the following significant findings.

The overall impact of random padding on HINT is minimal. As shown in Table 6, HINT's accuracy slightly decreases, accompanied by a marginal decrease in the TPR and a slight uptick in FPR. Additionally, baselines have been proven effective, particularly those leveraging packet size features. Models like MTBD and AppScanner, which derive insights from initial TLS handshake, maintain strong performance despite the padding. While TLSFP relies on encapsulated TLS handshake features, its performance is affected. GraphDAPP and HBA exhibit comparable prowess, with GraphDAPP boasting a high true positive rate, adept at capturing characteristics of the positive class. HBA is a host-based detection method, but limitations in DF's precision impede its performance.

6.3. Evaluate traffic shaping effects

In this section, we use the HTPA-TS-n dataset to validate the performance of HINT and benchmark algorithms. From Table 7, we derive several key insights:

HINT shows the greatest stability in all obfuscated datasets. As the number of decoy packets increases, HINT maintains a stable TPR with a slight rise in FPR, indicating that the interference caused by decoy packets is relatively limited. Upon analysis, we observed an increase in misclassified normal web services as tunnel services, from 8 to 25, when compared to results without traffic shaping. Meanwhile, the CNN–LSTM model shows a minor decline in performance with a gradual decrease in TPR as the decoy packets increase. We conjecture that reductions in the CNN–LSTM model's precision may indirectly affect HINT's performance.

Traffic shaping is a more effective obfuscation technique than random padding, as it effectively disrupts intrinsic patterns in packet sizes and timings. This disruption impacts all benchmark algorithms, leading to a notable decline in classification performance. Tree-based models, such as AppScanner, which rely heavily on packet size features, are especially vulnerable. Additionally, as the number of decoy packets increases, models like DF and HBA, which leverage neural networks to extract features and mitigate the impact of noise, experience an accuracy drop of approximately 5%. In contrast, TLSFP and GraphDAPP

Table 7
Comparison results on three dataset at flow level.

Method	HTPA-TS-1				HTPA-TS-2				HTPA-TS-3			
	ACC	TPR	FPR	AUC	ACC	TPR	FPR	AUC	ACC	TPR	FPR	AUC
AppScanner	0.9083	0.6704	0.0089	0.9401	0.8848	0.4579	0.011	0.9044	0.8473	0.3821	0.0101	0.8515
DF	0.8947	0.7922	0.0851	0.9320	0.8676	0.7925	0.1132	0.8978	0.8496	0.6357	0.0984	0.8346
GraphDAPP	0.8588	0.9778	0.1646	0.9760	0.8575	0.9698	0.1646	0.9742	0.8569	0.9662	0.1646	0.9710
MTBD	0.9295	0.6897	0.0007	0.9952	0.8924	0.5463	0.0011	0.9944	0.8819	0.4801	0.0012	0.9932
TLSFP	0.7043	0.9643	0.5573	0.7035	0.6903	0.9343	0.5924	0.6849	0.6735	0.9152	0.6847	0.6748
HBA	0.9038	0.8025	0.0684	0.9428	0.8823	0.8147	0.1084	0.9183	0.8528	0.7242	0.0964	0.8582
CNN–LSTM	0.9620	0.9376	0.0057	0.9752	0.9505	0.9129	0.0057	0.9709	0.9484	0.9039	0.0057	0.9658
HINT	**0.9981**	**0.9934**	**0.0011**	**0.9971**	**0.9979**	**0.9929**	**0.0015**	**0.9969**	**0.9963**	**0.9923**	**0.0018**	**0.9963**

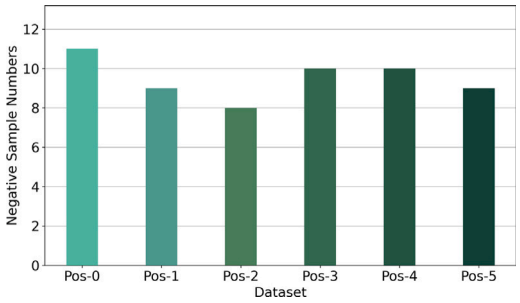


Fig. 4. Comparison of negative sample numbers across Pos-n datasets.

Table 8
Impact of decoy packet insertion at specific upstream positions on HINT's service-based performance across Pos-n datasets: Insertion at the 0th, 3rd, and 4th upstream positions in flow traces significantly impacts results.

Dataset	ACC	TPR	FPR	AUC
Pos-0	**0.9849**	**1.0000**	**0.0182**	**0.9923**
Pos-1	0.9876	1.0000	0.0137	0.9976
Pos-2	0.9896	1.0000	0.0115	0.9982
Pos-3	**0.9863**	**0.9868**	**0.0137**	**0.9973**
Pos-4	**0.9863**	**1.0000**	**0.0151**	**0.9979**
Pos-5	0.9876	1.0000	0.0137	0.9974

experience less significant performance drops. TLSFP's use of Chi-squared (χ^2) and Mahalanobis distance in its classifier constrains the interference from decoy packets. Meanwhile, GraphDAPP converts the problem of encrypted traffic classification into a graph classification problem, making it less susceptible to decoy packets disruptions.

HINT remains robust against variations in decoy packet insertion positions. In order to assess the effect of different insertion positions on HINT's accuracy, we conduct a series of experiments using the Pos-n datasets. We specifically evaluate HINT's performance using service-based evaluation metrics. As demonstrated in Table 8, the model exhibits robustness across various insertion points. Fig. 4 illustrates the count of misclassified services and highlights that positions such as the start or immediately after the third and fourth packets in upstream traffic significantly disrupt the model. Notably, these insertion points coincide with the initiation of both the outer and encapsulated TLS handshakes, implying their interference is more significant than at other positions.

6.4. Analysis of hierarchical attention

HINT adeptly integrates neighboring nodes and multiple meta-path relationships in a hierarchical mechanism. To facilitate deeper insight into its decision-making process, we have visualized the attention scores to illustrate the weights of neighbors and meta-paths.

Analysis on Node-Level Attention. As previously noted, our model is capable of calculating attention scores between a node and its adjacent nodes within any given meta-path. Neighbors that are pivotal for a specific task usually receive higher attention scores. To illustrate, we take a specific HTTPS tunnel service node s from the HTPA dataset. The attention scores between node s and its adjacent nodes over FSF, SHS, and CSC meta-paths were visualized. As depicted in Fig. 5(a), client count and information entropy garnered the most attention weight, followed by RTT ratio and pkts/bytes per flow. In Fig. 5(b), service nodes s and s_2 received higher attention scores, correlating with their larger flow counts. Conversely, as shown in Fig. 5(c), client nodes c_1 and c_2, with fewer than 20 flows, attracted lower attention scores. This analysis confirms that our model's node-level attention effectively differentiates among neighbors.

Analysis on Semantic-Level Attention. HINT is capable of evaluating the significance of meta-paths for particular tasks. Continuing with the node s, we calculated the attention values for the FSF, SHS, and CSC meta-paths across four datasets. The result is displayed in Fig. 6, attention values reveal a hierarchy of meta-path importance: SHS is the most critical, followed by CSC and FSF.

Table 9
Comparative results of ablation studies on host, service, and client features with respect to service-based performance across the HTPA dataset.

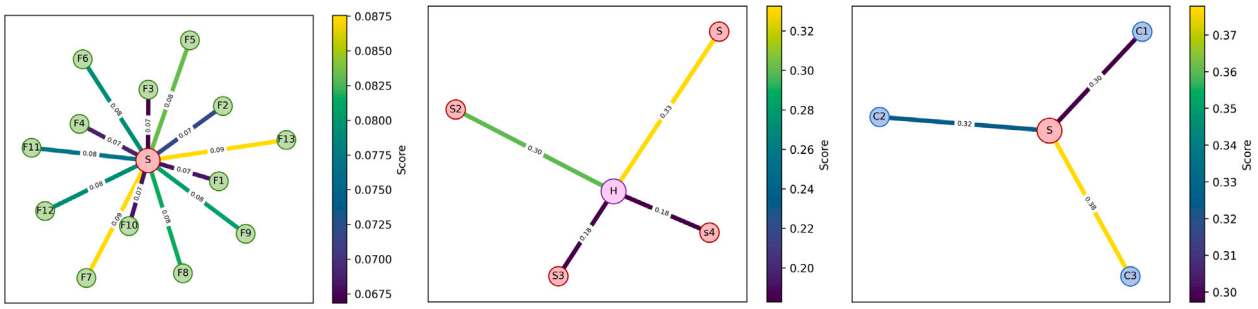
Model	ACC	TPR	FPR	AUC
w/ client	0.9838	0.9832	0.0029	0.9960
w/ service	0.6543	0.2300	0.0000	0.6904
w/ host	**0.9547**	**0.9215**	**0.0009**	**0.9996**
w/o client	**0.9767**	**0.9315**	**0.0013**	**0.9987**
w/o service	0.9840	0.9623	0.0009	0.9996
w/o host	0.9675	0.9932	0.0575	0.9931

6.5. Ablation study

To further evaluate the impact of features at the host, service, and client levels, we carry out a series of ablation studies, particularly by designing six contrast experiments on the HTPA dataset:

- **HINT w/ client**: The semantic-level attention focuses solely on learning the importance of CSC and FCF meta-paths, which are directly associated with client features.
- **HINT w/ service**: The semantic-level attention focuses solely on learning the importance of FSF meta-path, which is directly associated with service statistical characteristics.
- **HINT w/ host**: The semantic-level attention focuses solely on learning the importance of SHS meta-path, which is directly associated with host topological characteristics.
- **HINT w/o client**: The semantic-level attention focuses on learning the importance of SHS and FSF meta-paths.
- **HINT w/o service**: The semantic-level attention focuses on learning the importance of SHS, CSC, and FCF meta-paths.
- **HINT w/o host**: The semantic-level attention focuses on learning the importance of FSF, CSC, and FCF meta-paths.

As shown in Table 9, client features play a pivotal role in enhancing HINT's effectiveness, while host features are equally crucial.



(a) FSF attention of node s (b) SHS attention of node s (c) CSC attention of node s

Fig. 5. Meta-path based neighbors of node s and corresponding attention values.

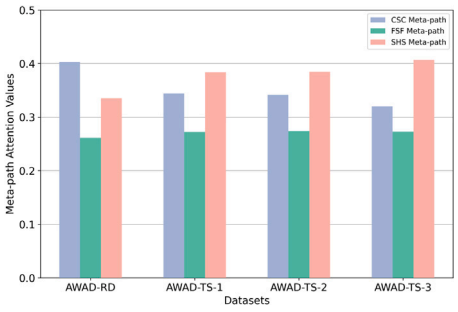


Fig. 6. Meta-path attention values on four datasets.

However, our findings also reveal a limitation. When HINT relies solely on service-level features (HINT w/ service), both the TPR and FPR are disappointingly low. This results in numerous HTTPS tunnel services being misclassified as normal HTTPS, revealing the inadequacy of just service-level features for reliable classification. Notably, host-level features contribute significantly to elevating HINT's performance. Specifically, HINT only with host-level features(HINT w/ host) stands out, achieving a perfect TPR and an impressive AUC. Concurrently, HINT without client-level features (HINT w/o client) still achieves near-perfect scores across all metrics, indicating that the combination of host-level and service-level features captures most of the necessary information for accurate classification. **This affirms HINT's ability to identify tunnel traffic with high accuracy, without relying on packet sequences or payload information from network traffic.**

6.6. Parameter analysis

In this section, we examine parameter sensitivity and present the classification outcomes of the HINT model on the HTPA dataset with different parameters, as shown in Fig. 7.

- **Number of Attention Heads.** To evaluate the effect of multi-head attention, we investigated the performance of HINT with varying numbers of attention heads, as depicted in Fig. 7(a). The results indicate that a greater number of attention heads tends to enhance performance. However, there is only a marginal improvement in HINT's performance with more than 8 attention heads. Additionally, we observed that multi-head attention contributes to the stability of the training process.
- **Dimension of Node Embedding.** As displayed in Fig. 7(b), performance appears to initially increase with the embedding dimension but then begins to plateau and slightly decrease. This trend suggests that HINT requires an optimal dimension to encode node information, and larger dimensions may lead to additional redundancy and unnecessary complexity.

Table 10
Time complexity of knowledge graph construction.

Node type	Label	Time complexity
Client Fingerprint	F_{c1}-F_{c4}	$O(N \log N)$
Client Fingerprint	F_{c5}	$O(\sum_{l=1}^{L}(N \times S \times Ci_l \times K_l^2 \times Co_l)) + O(N \times h^2)$
Service Fingerprint	F_{s1}-F_{s8}	$O(N)$
Service Fingerprint	F_{s9}-F_{s12}	$O(N)$
Service Fingerprint	F_{s13}	$O(N^2 \times S)$

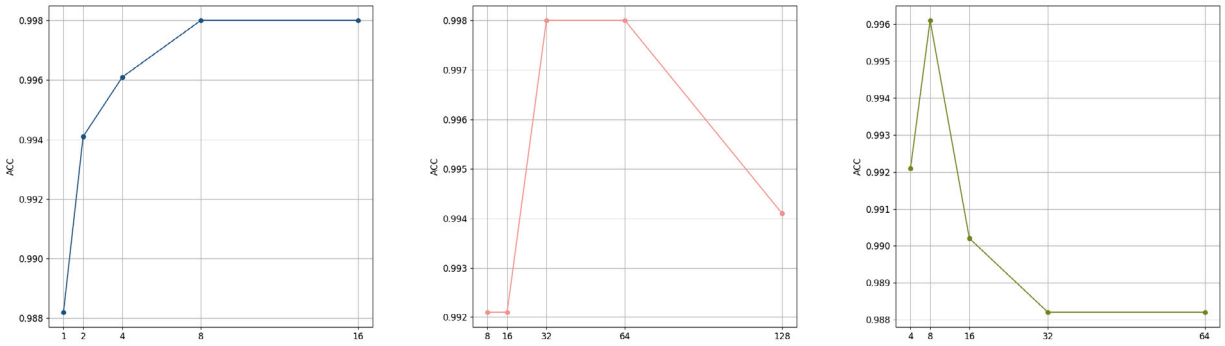
- **Dimension of Semantic-Level Attention Vector.** Determining the ideal size for the semantic-level attention vector is critical, as demonstrated in Fig. 7(c). We find that HINT's performance improves with an increase in the dimension of the semantic-level attention vector, peaking when the dimension is set to 8. Beyond this point, the performance starts to decline, likely due to overfitting, indicating that there is an optimal size for capturing complex relationships without compromising computational efficiency.

6.7. Time complexity

In this section, we evaluate the efficiency of the HINT method in the inference stage, which refers to the time spent on classifying an unknown flow with a well-trained classifier. The inference time consists of two main parts: the feature extraction time for constructing the knowledge graph and the prediction time for labeling flows. Table 10 outlines the time complexities associated with extracting fingerprint features from traffic data and encapsulating them as graph nodes. Here, N is the number of flows, S represents the number of packets per flow, Ci_l and Co_l are the input and output channels of the lth convolutional layer, K_l is the kernel size, and H is the number of hidden units in the LSTM network. Additionally, for prediction, the time complexity of dual-level attention mechanism is $O(M E_{\Phi} K)$, where M is the number of meta-paths, K is the number of attention heads, and E_{Φ} is the number of meta-path-based node pairs. Overall, the complexities grow linearly with the number of flows and node pairs.

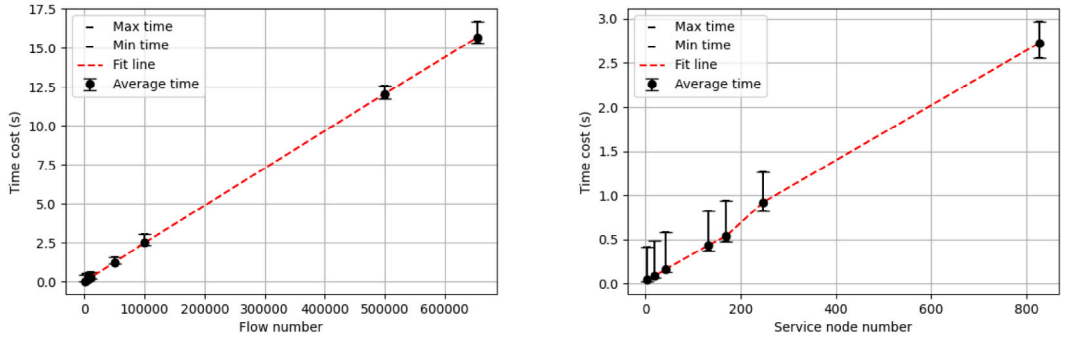
To provide deeper insights into time consumption, we conducted experiments on the HTPA dataset. As neural networks were used in both the Traffic Feature Extraction and Label Prediction stages, we measured the time required for each. Fig. 8(a) illustrates the time consumption of the CNN–LSTM model, which was used to extract traffic characteristics (e.g., Fc5). The computational complexity of the model increases linearly with the input flows. Similarly, Fig. 8(b) presents the time required for the dual-level attention in label prediction, where the average time also grows linearly with the number of service nodes. Overall, our experiments show that the CNN–LSTM model required less than 15 s to process up to 600,000 flows, while the dual-level attention required under 3 s for up to 800 service nodes.

In conclusion, the complexity of the HINT model is manageable. Additionally, the dual-level attention task can be parallelized across node



(a) Number of attention heads (b) Dimension of Node embedding (c) Dimension of semantic-level attention vector

Fig. 7. Sensitivity analysis of HINT with different parameters.



(a) The time complexity test of the CNN–LSTM model (b) The time complexity test of the dual-level attention

Fig. 8. Time complexity analysis in inference stage.

pairs and meta-paths, offering strong parallel processing capabilities. This positions the model for efficient deployment in large-scale data environments, and further optimization in hardware and algorithms is anticipated to enhance its performance.

7. Discussion

Our research identifies a critical oversight in previous HTTPS tunnel traffic classification studies, which have focused mainly on sequence features and payloads while neglecting host-level and service-level features. By analyzing host topology and service-level statistics features, we show that accurate classification is achievable, especially in VPN scenarios with multiple services on a single host. In the future, to enhance the classifier's performance, we will explore additional host and service level features by integrating information gathered from a series of flows over time.

Despite its strengths, the HINT method has certain limitations. **Environmental Constraint**: HINT relies on traffic characteristics, topology, and long-span statistics, making it well-suited for environments like data centers with access to global network traffic. Ablation studies show that the model's performance declines in the absence of comprehensive network data. **Multiplexing Constraint**: Connection reuse enables many sessions with different purposes to be multiplexed within a single TLS connection, complicating flow fingerprinting. Multiplexed connections are unsuitable for our algorithm, so we plan to explore techniques such as flow segmentation to improve accuracy. **Processing Constraint**: HINT calculates features over an extended observation period, limiting real-time applicability. Though the model is capable of periodic output (e.g., hourly), future research could focus on reducing this time window to enable near-real-time updates while preserving accuracy.

8. Conclusion

In this paper, we propose a novel method named HINT for detecting HTTPS tunnel traffic. HINT uses a knowledge graph to model intricate connections between clients and the VPN services. To enrich the graph's semantics, we incorporate novel characteristics that are challenging to disguise and encapsulate them into specialized fingerprint nodes. Then we apply a hierarchical attention mechanism to discern the importance of different node types and meta-paths. Finally, we implement node classification using a classifier. We evaluate our method on self-collected datasets, experimental results and extended analysis demonstrate that HINT maintains robust classification power even when traffic shaping and padding techniques are employed. Notably, it can achieve superior performance in identifying tunnel traffic without dependence on packet sequences or payload information.

CRediT authorship contribution statement

Mengyan Liu: Writing – original draft, Visualization, Software, Methodology, Conceptualization. **Gaopeng Gou:** Supervision, Project administration, Funding acquisition. **Gang Xiong:** Validation, Supervision, Resources. **Junzheng Shi:** Software, Resources, Methodology. **Zhong Guan:** Writing – review & editing, Validation. **Hanwen Miao:** Investigation, Formal analysis, Data curation. **Yang Li:** Writing – review & editing, Software, Data curation.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work is supported by National Key Laboratory of Science and Technology on Blind Signal Processing (No. 23007522).

Data availability

I have shared the link at

HTTPS Tunneling Predictive Analysis Dataset (Original data) (github)

References

[1] A. S, S. Sankaran, Cross protocol attack on ipsec-based VPN, in: 2023 11th International Symposium on Digital Forensics and Security, ISDFS, 2023, pp. 1–6, http://dx.doi.org/10.1109/ISDFS58141.2023.10131787.

[2] H. Abbas, N. Emmanuel, M.F. Amjad, T. Yaqoob, M. Atiquzzaman, Z. Iqbal, N. Shafqat, W.B. Shahid, A. Tanveer, U. Ashfaq, Security assessment and evaluation of VPNs: A comprehensive survey, ACM Comput. Surv. 55 (13s) (2023) http://dx.doi.org/10.1145/3579162.

[3] T. Bui, S. Rao, M. Antikainen, T. Aura, Client-side vulnerabilities in commercial vpns, in: Secure IT Systems: 24th Nordic Conference, NordSec 2019, Aalborg, Denmark, November 18–20, 2019, Proceedings 24, Springer, 2019, pp. 103–119, http://dx.doi.org/10.1007/978-3-030-35055-0_7.

[4] M. Finsterbusch, C. Richter, E. Rocha, J.-A. Muller, K. Hanssgen, A survey of payload-based traffic classification approaches, IEEE Commun. Surv. Tutor. 16 (2) (2014) 1135–1156, http://dx.doi.org/10.1109/SURV.2013.100613.00161.

[5] A. Azab, M. Khasawneh, S. Alrabaee, K.-K.R. Choo, M. Sarsour, Network traffic classification: Techniques, datasets, and challenges, Digit. Commun. Netw. 10 (3) (2024) 676–692, http://dx.doi.org/10.1016/j.dcan.2022.09.009.

[6] P. Gao, G. Li, Y. Shi, Y. Wang, VPN traffic classification based on payload length sequence, in: 2020 International Conference on Networking and Network Applications, NaNA, 2020, pp. 241–247, http://dx.doi.org/10.1109/NaNA51271.2020.00048.

[7] W3Techs, Usage statistics of default protocol https for websites, 2024, https://w3techs.com/technologies/breakdown/ce-httpsdefault/ranking. (Accessed 16 June 2024).

[8] S. Miller, K. Curran, T. Lunney, Multilayer perceptron neural network for detection of encrypted vpn network traffic, in: 2018 International Conference on Cyber Situational Awareness, Data Analytics and Assessment (Cyber SA), 2018, pp. 1–8, http://dx.doi.org/10.1109/CyberSA.2018.8551395.

[9] S. Miller, K. Curran, T. Lunney, Detection of anonymising proxies using machine learning, Int. J. Digit. Crime Forensics (IJDCF) 13 (6) (2021) 1–17, http://dx.doi.org/10.4018/IJDCF.286756.

[10] Y. Zhang, W. Sun, S. Zhang, Identify VPN traffic under HTTPS tunnel using three-dimensional sequence features, in: Proceedings of the 2022 11th International Conference on Networks, Communication and Computing, ICNCC '22, Association for Computing Machinery, New York, NY, USA, 2023, pp. 18–23, http://dx.doi.org/10.1145/3579895.3579899.

[11] C. Wang, J. Yin, Z. Li, H. Xu, Z. Zhang, Q. Liu, Identifying VPN servers through graph-represented behaviors, in: Proceedings of the ACM Web Conference 2024, WWW '24, Association for Computing Machinery, New York, NY, USA, 2024, pp. 1790–1799, http://dx.doi.org/10.1145/3589334.3645552.

[12] R. Wails, G.A. Sullivan, M. Sherr, R. Jansen, On precisely detecting censorship circumvention in real-world networks, in: Network and Distributed System Security, 2024, http://dx.doi.org/10.14722/ndss.2024.23394.

[13] S. Frolov, E. Wustrow, The use of TLS in censorship circumvention, in: Network and Distributed System Security, 2019, http://dx.doi.org/10.14722/ndss.2019.23511.

[14] D. Xue, M. Kallitsis, A. Houmansadr, R. Ensafi, Fingerprinting obfuscated proxy traffic with encapsulated TLS handshakes, in: 33rd USENIX Security Symposium, USENIX Security 24, USENIX Association, Philadelphia, PA, 2024, pp. 2689–2706.

[15] Y. Wang, P. Ji, B. Ye, P. Wang, R. Luo, H. Yang, Gohop: Personal VPN to defend from censorship, in: 16th International Conference on Advanced Communication Technology, 2014, pp. 27–33, http://dx.doi.org/10.1109/ICACT.2014.6778916.

[16] W. Xu, F. Zou, Obfuscated tor traffic identification based on sliding window, Secur. Commun. Netw. 2021 (1) (2021) 5587837, http://dx.doi.org/10.1155/2021/5587837.

[17] N. Rust-Nguyen, S. Sharma, M. Stamp, Darknet traffic classification and adversarial attacks using machine learning, Comput. Secur. 127 (2023) 103098, http://dx.doi.org/10.1016/j.cose.2023.103098.

[18] B. Wang, Y. Guan, G. Gou, P. Fu, Z. Li, Q. Yang, C. Liu, MTBD: HTTPS tunnel detection based on multi-dimension traffic behaviors decision, in: 2022 IEEE 24th Int Conf on High Performance Computing & Communications, IEEE, 2022, pp. 474–481, http://dx.doi.org/10.1109/HPCC-DSS-SmartCity-DependSys57074.2022.00091.

[19] A.J. Pinheiro, J.M. Bezerra, D.R. Campelo, Packet padding for improving privacy in consumer IoT, in: 2018 IEEE Symposium on Computers and Communications, ISCC, 2018, pp. 925–929, http://dx.doi.org/10.1109/ISCC.2018.8538744.

[20] S. Xiong, A.D. Sarwate, N.B. Mandayam, Defending against packet-size side-channel attacks in iot networks, in: 2018 IEEE International Conference on Acoustics, Speech and Signal Processing, ICASSP, 2018, pp. 2027–2031, http://dx.doi.org/10.1109/ICASSP.2018.8461330.

[21] A. Engelberg, A. Wool, Classification of encrypted IoT traffic despite padding and shaping, in: Proceedings of the 21st Workshop on Privacy in the Electronic Society, WPES '22, Association for Computing Machinery, New York, NY, USA, 2022, pp. 1–13, http://dx.doi.org/10.1145/3559613.3563191.

[22] N. Apthorpe, D.Y. Huang, D. Reisman, A. Narayanan, N. Feamster, Keeping the smart home private with smart(er) IoT traffic shaping, Proc. Privacy Enhancing Technol. 2019 (3) (2019) 128–148, http://dx.doi.org/10.2478/popets-2019-0040.

[23] M. Zain ul Abideen, S. Saleem, M. Ejaz, Vpn traffic detection in ssl-protected channel, Secur. Commun. Netw. 2019 (2019) 1–17, http://dx.doi.org/10.1155/2019/7924690.

[24] A. Abimbola, J. Munoz, W.J. Buchanan, NetHost-sensor: Investigating the capture of end-to-end encrypted intrusive data, Comput. Secur. 25 (6) (2006) 445–451.

[25] G. He, M. Yang, X. Gu, J. Luo, Y. Ma, A novel active website fingerprinting attack against tor anonymous system, in: Proceedings of the 2014 IEEE 18th International Conference on Computer Supported Cooperative Work in Design, CSCWD, 2014, pp. 112–117, http://dx.doi.org/10.1109/CSCWD.2014.6846826.

[26] J. Amann, R. Sommer, Exploring tor's activity through long-term passive TLS traffic measurement, in: Passive and Active Measurement: 17th International Conference, PAM 2016, Heraklion, Greece, March 31-April 1, 2016. Proceedings 17, 2016, pp. 3–15, http://dx.doi.org/10.1007/978-3-319-30505-9_1.

[27] V.V. Lapshichyov, TLS certificates of the tor network and their distinctive features, Int. J. Syst. Softw. Secur. Prot. (IJSSSP) 10 (2) (2019) 20–43, http://dx.doi.org/10.4018/IJSSSP.2019070102.

[28] A. Habibi Lashkari, G. Draper Gil, M.S.I. Mamun, A.A. Ghorbani, Characterization of tor traffic using time based features, in: Proceedings of the 3rd International Conference on Information Systems Security and Privacy, 2017, pp. 253–262, http://dx.doi.org/10.5220/0006105602530262.

[29] Z. Yao, J. Ge, Y. Wu, X. Zhang, Q. Li, L. Zhang, Z. Zou, Meek-based tor traffic identification with hidden markov model, in: 2018 IEEE 20th International Conference on High Performance Computing and Communications, IEEE, 2018, pp. 335–340.

[30] W. Sun, Y. Zhang, J. Li, C. Sun, S. Zhang, A deep learning-based encrypted VPN traffic classification method using packet block image, Electronics 12 (1) (2022) 115.

[31] L. Guo, Q. Wu, S. Liu, M. Duan, H. Li, J. Sun, Deep learning-based real-time VPN encrypted traffic identification methods, J. Real-Time Image Process. 17 (1) (2020) 103–114.

[32] D. Sarkar, P. Vinod, S.Y. Yerima, Detection of tor traffic using deep learning, in: 2020 IEEE/ACS 17th International Conference on Computer Systems and Applications, AICCSA, 2020, pp. 1–8, http://dx.doi.org/10.1109/AICCSA50499.2020.9316533.

[33] Y. He, L. Hu, R. Gao, Detection of tor traffic hiding under Obfs4 protocol based on two-level filtering, in: 2019 2nd International Conference on Data Intelligence and Security, ICDIS, 2019, pp. 195–200, http://dx.doi.org/10.1109/ICDIS.2019.00036.

[34] O. Salman, I.H. Elhajj, A. Kayssi, A. Chehab, Mutated traffic detection and recovery: an adversarial generative deep learning approach, Ann. Telecommun. 77 (5) (2022) 395–406, http://dx.doi.org/10.1007/s12243-022-00909-8.

[35] T. Yang, R. Jiang, H. Deng, Q. Li, Z. Liu, AE-DTI: An efficient darknet traffic identification method based on autoencoder improvement, Appl. Sci. 13 (16) (2023) 9353, http://dx.doi.org/10.3390/app13169353.

[36] S. Al-E'mari, Y. Sanjalawe, S. Fraihat, Detection of obfuscated tor traffic based on bidirectional generative adversarial networks and vision transform, Comput. Secur. 135 (2023) 103512, http://dx.doi.org/10.1016/j.cose.2023.103512.

[37] O. Salman, I.H. Elhajj, A. Kayssi, A. Chehab, Denoising adversarial autoencoder for obfuscated traffic detection and recovery, in: International Conference on Machine Learning for Networking, Springer, 2019, pp. 99–116, http://dx.doi.org/10.1007/978-3-030-45778-5_8.

[38] X. Wang, H. Ji, C. Shi, B. Wang, Y. Ye, P. Cui, P.S. Yu, Heterogeneous graph attention network, in: The World Wide Web Conference, WWW '19, Association for Computing Machinery, New York, NY, USA, 2019, pp. 2022–2032, http://dx.doi.org/10.1145/3308558.3313562.

[39] E. Chiapponi, M. Dacier, O. Thonnard, M. Fangar, V. Rigal, BADPASS: Bots taking advantage of proxy as a service, in: Information Security Practice and Experience: 17th International Conference, ISPEC 2022, Taipei, Taiwan, November 23-25, 2022, Proceedings, Springer-Verlag, Berlin, Heidelberg, 2022, pp. 327–344, http://dx.doi.org/10.1007/978-3-031-21280-2_18.

[40] Y. Sun, J. Han, X. Yan, P.S. Yu, T. Wu, Pathsim: Meta path-based top-k similarity search in heterogeneous information networks, Proc. VLDB Endow. 4 (11) (2011) 992–1003.

[41] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A.N. Gomez, Ł. Kaiser, I. Polosukhin, Attention is all you need, in: Proceedings of the 31st International Conference on Neural Information Processing Systems, NIPS '17, Curran Associates Inc., Red Hook, NY, USA, 2017, pp. 6000–6010.

[42] Seakfind, XTLS vision, 2024, https://github.com/seakfind/examples/blob/main/xtls-vision/README.md. (Accessed 22 July 2024).

[43] V.F. Taylor, R. Spolaor, M. Conti, I. Martinovic, AppScanner: Automatic fingerprinting of smartphone apps from encrypted network traffic, in: 2016 IEEE European Symposium on Security and Privacy (EuroS&P), 2016, pp. 439–454, http://dx.doi.org/10.1109/EuroSP.2016.40.

[44] P. Sirinam, M. Imani, M. Juarez, M. Wright, Deep fingerprinting: Undermining website fingerprinting defenses with deep learning, in: Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security, CCS '18, Association for Computing Machinery, New York, NY, USA, 2018, pp. 1928–1943, http://dx.doi.org/10.1145/3243734.3243768.

[45] M. Shen, J. Zhang, L. Zhu, K. Xu, X. Du, Accurate decentralized application identification via encrypted traffic analysis using graph neural networks, IEEE Trans. Inf. Forensics Secur. 16 (2021) 2367–2380.

[46] D. Kingma, J. Ba, Adam: A method for stochastic optimization, in: International Conference on Learning Representations, 2014, http://dx.doi.org/10.48550/arXiv.1412.6980.



Mengyan Liu received her master's degree from Beijing University of Posts and Telecommunications, China, in 2018. She is currently a graduate student studying for a PhD in the Institute of Information Engineering, Chinese Academy of Sciences, China. Her research interests include network security, user privacy and network behavior analysis.



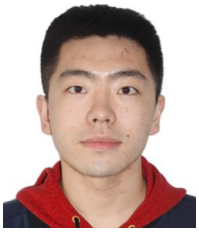
Gaopeng Gou received the M.Eng. degree from Beihang University, China, in 2008, and the joint Ph.D. degree from University of Alberta and Beihang University, in 2011 and 2014, respectively. He is a Senior Engineer with the Ranking of Full Professor and a doctoral supervisor in the Institute of Information Engineering, Chinese Academy of Sciences, China. His research interests include cyber security and network traffic analysis.



Gang Xiong is currently a Full Professor and Ph.D. Supervisor with the Institute of Information Engineering, Chinese Academy of Sciences, China. He has authored more than 100 papers in refereed journals and conference proceedings. His research interests include network and information security. He is a member of the 3rd Communication Security Technical Committee of China Institute of Communications.



Junzheng Shi received the M.Eng. degree from Beijing University of Posts and Telecommunications, China, in 2013, and the Ph.D. degree from University of Chinese Academy of Sciences in 2021. He is currently a Senior Engineer with the Institute of Information Engineering, Chinese Academy of Sciences, China. His research interests include network measurement and behavior analysis.



Zhong Guan received the Ph.D. degree in information security from the University of Chinese Academy of Sciences in 2023. He is currently a research assistant with the Institute of Information Engineering, Chinese Academy of Sciences, China. His research interests include network security and privacy.



Hanwen Miao received his master's degree from University of Michigan in 2020. He is currently a Ph.D. student at the Institute of Information Engineering, Chines Academy of Sciences. His research interests include network traffic analysis and artificial intelligence.



Yang Li received the B.Eng. degree from Henan University, China, in 2022. He is currently a graduate student studying for a Ph.D. in the Institute of Information Engineering, Chinese Academy of Sciences, China. His research interests include IPv6 privacy and security.